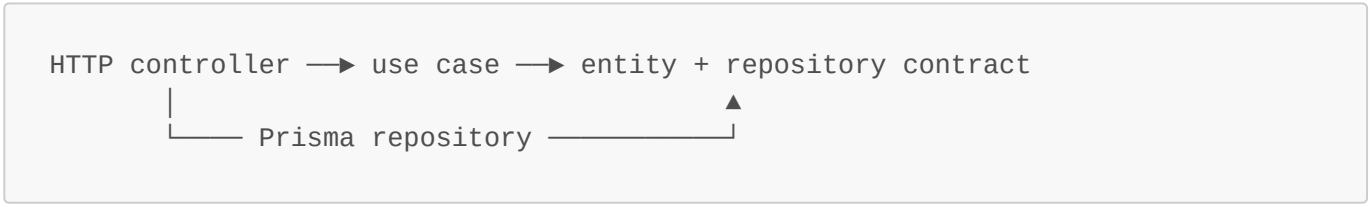
Clean Architecture with Nest.js

Objective

This repository separates business rules from HTTP, NestJS, Prisma, PostgreSQL, and Vault. The `users` module is the working example: a controller receives an HTTP request, a use case applies the application rule, and a Prisma repository performs persistence through a domain-facing repository contract.

The practical rule is that code closer to the business should know less about frameworks and delivery mechanisms.



`UsersModule` connects these pieces. It is the place where the repository contract is bound to the Prisma implementation.

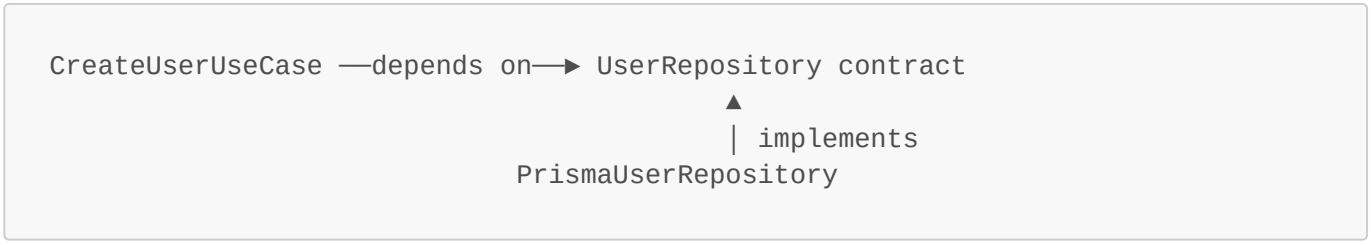
The idea: policy and details

Clean Architecture distinguishes between **policy** and **details**.

- Policy is why the application exists: creating a user, preventing duplicate emails, and keeping a user's name and email valid.
- Details are how the policy is delivered or stored: an HTTP controller, Nest dependency injection, Prisma queries, PostgreSQL, and Vault.

The policy must not need to change merely because a detail changes. For example, `User.create()` validates the name and email without knowing whether the input was received from an HTTP request or whether the user will be stored by Prisma. Similarly, `CreateUserUseCase` asks for `UserRepository`; it does not issue a Prisma query itself.

This is dependency inversion in the current users module:



The use case owns the policy-facing dependency (`UserRepository`). The Prisma repository is the technical implementation that satisfies it. `UsersModule` selects the implementation at application startup.

How the current implementation is wired

Nest creates the running object graph in this order:

```
main.ts
  └── bootstrapConfig()
        └── VaultService.sync() loads validated secrets into process.env
  └── NestFactory.create(AppModule)
        └── AppModule imports UsersModule
              └── UsersModule imports PrismaModule
                    └── PrismaModule provides PrismaService
              └── USER_REPOSITORY is bound to PrismaUserRepository
                    └── PrismaUserRepository receives PrismaService
              └── user use cases receive USER_REPOSITORY
              └── UsersController receives the user use cases
```

The binding in `users.module.ts` is the key line of the architecture:

```
{
  provide: USER_REPOSITORY,
  useClass: PrismaUserRepository,
}
```

`CreateUserUseCase`, `GetUserByIdUseCase`, `GetAllUsersUseCase`, `UpdateUserUseCase`, and `DeleteUserUseCase` request `USER_REPOSITORY` in their constructors. Nest injects the `PrismaUserRepository` instance because of this module binding. The use cases therefore use the repository contract, while the module chooses Prisma as the implementation.

How a user creation request works

The following is the full flow of a successful `POST /api/v1/users` request:

1. `main.ts` has registered Nest's global `ValidationPipe`.
2. Nest matches the request to `UsersController.createUser`.
3. `CreateUserRequestDto` checks that `name` is a string with the configured length and that `email` has an email format.
4. The controller passes only `name` and `email` to `CreateUserUseCase.execute`; it does not pass the HTTP request object.
5. `CreateUserUseCase` calls `UserRepository.findByEmail` to prevent a duplicate email.
6. The use case calls `User.create`, which trims and validates the name/email, creates timestamps, and constructs the domain entity.
7. The use case calls `UserRepository.create`.
8. Nest has supplied `PrismaUserRepository` for that contract, so its `create` method calls `PrismaService.user.create`.
9. Prisma writes the database row. `PrismaUserRepository.toDomain` converts the returned row back into a `User` entity.
10. The controller wraps the result using `successResponse`, which produces `{ success, message, data }` JSON.

The same separation applies to read and update requests. The controller handles HTTP, the use case handles the action, `User` handles its invariant rules, and the Prisma repository handles persistence.

How to decide where existing code belongs

When changing this repository, use the question below to select the existing location.

If the change is about...	Put it in the current code that owns it
A user rule, such as normalizing an email or changing a name	`modules/users/domain/entities/user.entity.ts`
The sequence of an action, such as checking an existing user before create	The matching file in `modules/users/application/use-cases/`
The operations a use case needs from storage	`modules/users/domain/repositories/user.repository.ts`
A Prisma query or conversion between a Prisma record and `User`	`modules/users/infrastructure/persistence/prisma-user.repository.ts`
HTTP parameters, body validation, route handling, or JSON success wrapping	The current users controller or HTTP DTO files
Database-client lifecycle and PostgreSQL adapter construction	`infrastructure/database/prisma/prisma.service.ts`
Vault, startup, CORS, global prefix, versioning, or shutdown behavior	The matching file under `config/` or `main.ts`
Reusable HTTP/Nest behavior not owned by users	The matching file under `common/`
A reusable enum, type, constant, or utility	The matching file under `shared/`

Current source structure

```
src/                                    # Application source code
├── main.ts
├── app.module.ts
├── app.controller.ts
├── app.controller.spec.ts
├── app.service.ts
├── common/                              # Nest/HTTP cross-cutting code
│   ├── exceptions/                       # Application exception classes
│   │   └── unauthorized.exception.ts
│   ├── filters/                          # HTTP exception handling
│   │   └── http-exception.filter.ts
│   ├── guards/                           # Authentication and authorization
checks
```

```
│   │   ├── auth.guard.ts
│   │   └── roles.guard.ts
│   ├── interceptors/                       # Request/response interception
│   │   └── response.interceptor.ts
│   ├── middleware/                         # HTTP middleware
│   │   └── request-logger.middleware.ts
│   └── pipes/                              # Request validation/transformation
│       └── zod-validation.pipe.ts
├── config/                                 # Runtime configuration and
lifecycle code
│   ├── env/                                # Environment-variable validation
│   │   └── env.schema.ts
│   ├── lifecycle/                          # Startup and shutdown behavior
│   │   ├── bootstrap-config.ts
│   │   └── shutdown.service.ts
│   └── vault/                              # Vault secret retrieval and
validation
│       ├── vault.constants.ts
│       ├── vault.schema.ts
│       ├── vault.service.spec.ts
│       ├── vault.service.ts
│       └── vault.types.ts
├── infrastructure/                         # Shared technical integrations
│   └── database/                           # Database infrastructure
│       └── prisma/                         # Prisma client lifecycle and Nest
module
│           ├── prisma.module.ts
│           ├── prisma.service.spec.ts
│           └── prisma.service.ts
├── modules/                                # Business features grouped by
capability
│   ├── health/                             # Health-check feature
│   │   ├── health.controller.spec.ts
│   │   ├── health.controller.ts
│   │   └── health.module.ts
│   └── users/                              # User-management feature
│       ├── users.module.ts
│       ├── domain/                         # User business rules and contracts
│       │   ├── entities/                   # User domain entity
│       │   │   └── user.entity.ts
│       │   └── repositories/               # Storage contract required by
users
│       │       └── user.repository.ts
│       ├── application/                    # User actions and use-case
input/output
│       │   ├── dto/                        # Data accepted/returned by user
use cases
│       │   │   ├── create-user.input.ts
│       │   │   ├── update-user.input.ts
│       │   │   └── user.output.ts
│       │   └── use-cases/                  # Create, read, update, and delete
actions
│       │       ├── create-user.use-case.ts
│       │       ├── delete-user.use-case.ts
│       │       ├── get-all-users.use-case.ts
│       │       ├── get-user-by-id.use-case.ts
```

```
│       │       └── update-user.use-case.ts
│       ├── infrastructure/                # Users-specific technical adapters
│       │   └── persistence/               # User storage implementation
│       │       └── prisma-user.repository.ts
│       └── presentation/                  # User-facing delivery code
│           └── http/                      # HTTP routes and HTTP data
contracts
│               ├── controllers/           # Users HTTP controller and unit
test
│               │   ├── users.controller.spec.ts
│               │   └── users.controller.ts
│               └── dto/                   # HTTP request and response DTOs
│                   ├── requests/          # Request-body validation DTOs
│                   │   ├── create-user.request.ts
│                   │   └── update-user.request.ts
│                   └── responses/         # Public HTTP response DTOs
│                       └── user.response.ts
└── shared/                                 # Small reusable, framework-
neutral code
    ├── constants/                          # Shared application and seed
constants
    │   ├── app.constants.ts
    │   └── seed.constants.ts
    ├── enums/                              # Shared user enum values
    │   ├── user-role.enum.ts
    │   └── user-status.enum.ts
    ├── interfaces/                         # Shared TypeScript interfaces
    │   └── api-response.interface.ts
    ├── types/                              # Shared TypeScript types
    │   └── nullable.type.ts
    └── utils/                              # Shared helper functions
        ├── date.util.ts
        └── jwt.util.ts
```

Application bootstrap and shared technical code

File	Use in this project
`src/main.ts`	Starts Nest, validates/loads configuration, sets the `api` prefix and URI versioning, configures security, body parsing, CORS, and the listening port.
`src/app.module.ts`	Root Nest module. Registers configuration, health, users, Prisma, and shutdown support.
`src/app.controller.ts`	Provides the root `GET` endpoint.
`src/app.service.ts`	Supplies the response used by `AppController`.

File	Use in this project
`src/config/env/env.schema.ts`	Validates the Vault connection environment values.
`src/config/lifecycle/bootstrap-config.ts`	Validates environment configuration, then synchronizes Vault secrets before the Nest app is created.
`src/config/lifecycle/shutdown.service.ts`	Receives Nest shutdown notifications and logs the lifecycle event.
`src/config/vault/vault.service.ts`	Fetches, validates, and copies Vault secrets into `process.env`.
`src/config/vault/vault.schema.ts`	Defines the Vault secret shape currently used by the application.
`src/config/vault/vault.constants.ts`	Lists the known Vault secret keys.
`src/config/vault/vault.types.ts`	Types the Vault HTTP response.
`src/infrastructure/database/prisma/prisma.service.ts`	Creates the Prisma 7 client with the PostgreSQL adapter and owns connect/disconnect lifecycle calls.
`src/infrastructure/database/prisma/prisma.module.ts`	Exports `PrismaService` so feature infrastructure can use it.

Common and shared code

`common/` contains Nest/HTTP cross-cutting components. `shared/` contains small reusable types, enums, constants, and utilities. Neither folder owns user-specific business rules.

File	Use in this project
`common/exceptions/unauthorized.exception.ts`	Defines the application's unauthorized exception.
`common/filters/http-exception.filter.ts`	Reserved for HTTP exception-to-response handling.
`common/guards/auth.guard.ts`	Reserved authentication guard.
`common/guards/roles.guard.ts`	Reserved role authorization guard.
`common/interceptors/response.interceptor.ts`	Reserved response interception logic.
`common/middleware/request-logger.middleware.ts`	Reserved request logging middleware.
`common/pipes/zod-validation.pipe.ts`	Reserved Zod-backed validation pipe.
`shared/constants/app.constants.ts`	Application-wide constants, including the default port.
`shared/constants/seed.constants.ts`	Constants for database seed data.

File	Use in this project
`shared/enums/user-role.enum.ts`	The current user-role values.
`shared/enums/user-status.enum.ts`	The current user-status values.
`shared/interfaces/api-response.interface.ts`	Defines `ApiSuccessResponse`, `ApiErrorResponse`, and `successResponse`.
`shared/types/nullable.type.ts`	Reusable nullable type.
`shared/utils/date.util.ts`	Date utility functions.
`shared/utils/jwt.util.ts`	JWT utility functions.

Users module

`domain`

The domain is the business model. It contains no Prisma or HTTP imports.

File	Use in this project
`domain/entities/user.entity.ts`	Holds user state, validates names/emails when creating or changing a user, and exposes behavior such as `changeName` and `changeEmail`. `restore` rebuilds an entity from stored data without treating it as a new user.
`domain/repositories/user.repository.ts`	Defines the `UserRepository` contract used by the use cases: create, update, find, list, and delete. It also exports the `USER_REPOSITORY` injection token.

`application`

The application layer coordinates a single operation using the domain entity and the repository contract.

File	Use in this project
`application/dto/create-user.input.ts`	Input shape accepted by `CreateUserUseCase`.
`application/dto/update-user.input.ts`	Input shape accepted by `UpdateUserUseCase`.
`application/dto/user.output.ts`	Declares the application user output shape.
`application/use-cases/create-user.use-case.ts`	Checks email uniqueness, creates a `User` entity with a UUID, and saves it through `UserRepository`.
`application/use-cases/get-user-by-id.use-case.ts`	Finds a user by ID and fails when the user is absent.
`application/use-cases/get-all-users.use-case.ts`	Returns all users from the repository.
`application/use-cases/update-user.use-case.ts`	Finds a user, checks email uniqueness when necessary, applies entity behavior, and persists the result.

File	Use in this project
`application/use-cases/delete-user.use-case.ts`	Deletes a user through the repository contract.

`infrastructure`

File	Use in this project
`infrastructure/persistence/prisma-user.repository.ts`	Implements `UserRepository` with Prisma. It translates Prisma records into `User` entities in `toDomain`, and translates entity data into Prisma create/update commands.

The Prisma repository is allowed to import `PrismaService` because it is an infrastructure adapter. The domain entity and use cases are not allowed to import `PrismaService`.

`presentation/http`

File	Use in this project
`presentation/http/controllers/users.controller.ts`	Defines the users HTTP endpoints. It maps request DTO fields into use-case inputs and wraps results with `successResponse`.
`presentation/http/dto/requests/create-user.request.ts`	Validates `POST /users` data with `class-validator`: a name and email.
`presentation/http/dto/requests/update-user.request.ts`	Validates `PATCH /users/:id` data with `class-validator`: a name and email.
`presentation/http/dto/responses/user.response.ts`	Declares the current public user response type.

`users.module.ts`

`UsersModule` registers the five user use cases and `UsersController`. It binds the `USER_REPOSITORY` contract to `PrismaUserRepository`. This is the concrete dependency decision that lets use cases depend on a repository contract rather than Prisma.

Current request flow

For `POST /api/v1/users`:

```
CreateUserRequestDto
        | Nest validates name and email
        ▼
UsersController.createUser()
        | { name, email }
        ▼
CreateUserUseCase.execute()
```

```
        | UserRepository.findByEmail(), User.create(), UserRepository.create()
        ▼
PrismaUserRepository
        | PrismaService.user.create()
        ▼
PostgreSQL
```

The same boundaries apply to the other registered controller routes:

Method	Route	Controller method	Use case
`POST`	`/api/v1/users`	`createUser`	`CreateUserUseCase`
`GET`	`/api/v1/users/:id`	`getUserById`	`GetUserByIdUseCase`
`GET`	`/api/v1/users`	`getAllUsers`	`GetAllUsersUseCase`
`PATCH`	`/api/v1/users/:id`	`updateUser`	`UpdateUserUseCase`
`GET`	`/v1/health`	`checkHealth`	None; returns the health status directly.

`DeleteUserUseCase` is registered in `UsersModule`, but no delete controller route is currently defined.

Dependency rules used here

1. `User` and `UserRepository` must not import Prisma, PostgreSQL, HTTP DTOs, or controller code.
2. `PrismaUserRepository` implements the repository contract; it must keep Prisma record details inside the infrastructure layer.
3. `UsersController` uses request DTOs and calls use cases; it must not query Prisma directly.
4. `UsersModule` is the feature's composition root and is the only users file that binds `USER_REPOSITORY` to `PrismaUserRepository`.
5. `main.ts` and `app.module.ts` are application composition/bootstrap code, not places for user business rules.

Tests currently present

Test file	Scope
`src/app.controller.spec.ts`	Root controller behavior.
`src/config/vault/vault.service.spec.ts`	Vault service behavior.
`src/infrastructure/database/prisma/prisma.service.spec.ts`	Prisma service lifecycle behavior.
`src/modules/health/health.controller.spec.ts`	Health controller behavior.

Test file	Scope
`src/modules/users/presentation/http/controllers/users.controller.spec.ts`	Users controller construction with mocked use cases.
`test/app.e2e-spec.ts`	Application end-to-end test configuration.

Other project files

File or directory	Use in this project
`prisma/schema.prisma`	Prisma datasource, client generator, and `User` database model.
`prisma/migrations/`	Prisma migration history.
`generated/prisma/`	Generated Prisma client consumed by `PrismaService`; it is regenerated rather than hand-edited.
`prisma.config.ts`	Prisma CLI configuration, including schema and migration paths.
`test/jest-e2e.json`	Jest configuration for end-to-end tests.
`package.json`	Scripts, package dependencies, and Jest alias configuration.
`tsconfig.json`	TypeScript compiler configuration.
`nest-cli.json`	Nest CLI configuration.
`Dockerfile` and `docker.compose.yml`	Container build and local container orchestration configuration.
`.github/workflows/`	CI and deployment workflow definitions.
`lefthook.yml`	Git hook configuration.
`biome.json`	Biome formatter and linter configuration.
`sonar-project.properties`	SonarQube analysis configuration.